

QUERY CARTEL · CLICK-A-THON 2026 · ATLYS TRACK

Analytics Copilot

Feature spec in. Production schema + PM-ready insights out.

Agents instrument, analyze, and explain — fully traced.

Collapse weeks of instrumentation handoffs into one chat:
schema designed, context kept fresh, insights with charts & tables — every step traced.

Feature shipping outpaces analytics

Manual loop

Tracking PRD → schema → analysis takes weeks.
Context dies in handoffs.

Stale insight

By the time numbers land, the team has moved on.
No why, only what.

Unseen-spec day

Day-2 sealed feature must run through the same pipeline — no hardcoding.

Goal: collapse the loop — instrument, analyze, explain — on ClickHouse

1 Instrumentation

Spec → production DDL
(ORDER BY, partition, TTL, MVs)

2 Analytics

SQL-backed insights a PM would act on

3 Context

Living business layer; fresh after every table

4 Trace + UI

Langfuse proof + charts, tables, diffs

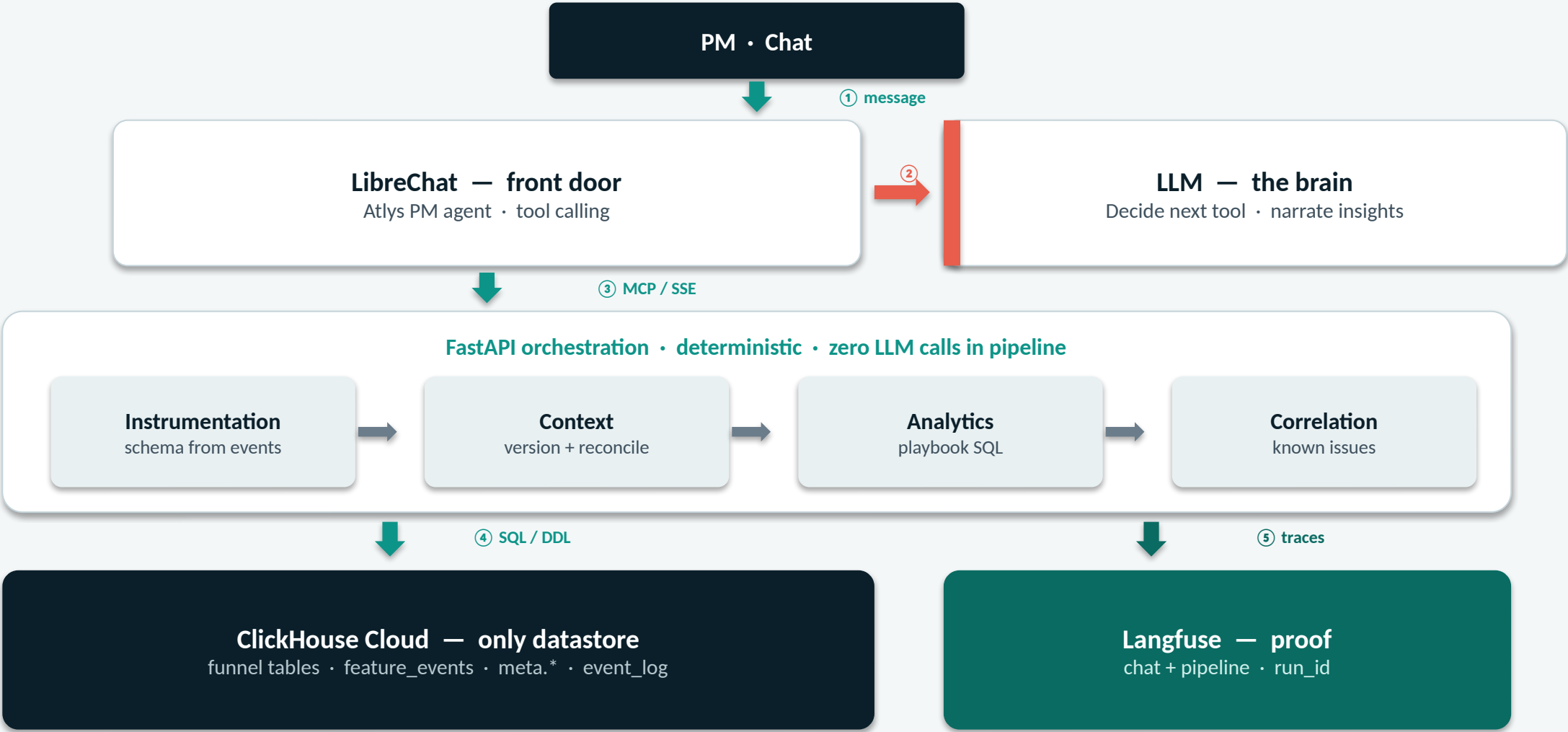
Feature spec in. Schema + insight out.

Agents instrument, analyze, and explain — fully traced. Charts & tables included.



Deterministic-first	Pipeline owns schema & numbers with zero LLM calls. The LLM is the brain for chat & narration — ClickHouse owns the math.
Compute in ClickHouse	Playbook SQL pushes aggregates into CH. The brain never sees raw rows — protects token budget & accuracy.
Generic for Day 2	Content-driven pipeline. No feature names hardcoded. Same chat flow for the sealed sixth spec.

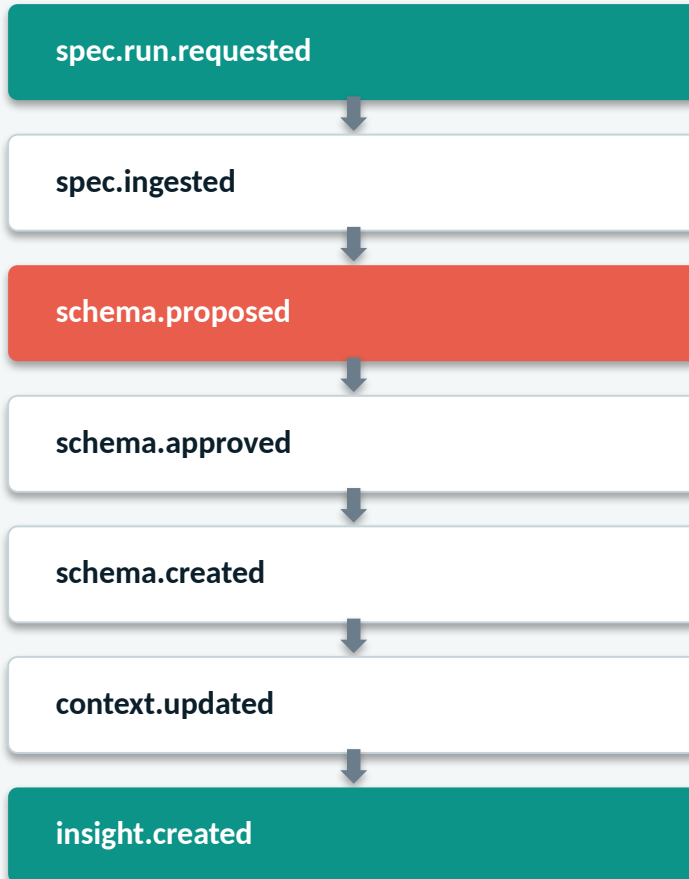
Architecture



⑧ Dashboard REST ← FastAPI · React shell shows insights, charts & tables, schema timeline, context diffs

Event-driven pipeline

Agents are event handlers. `atlys.event_log` is the durable backbone.



Why this design

Approval gate

Pauses at `schema.proposed`. Pending run in `meta.pending_runs`. DDL only after `approve_schema`.

Idempotent & auditable

Every transition → `event_log` row + Langfuse span. Replay the full chain from one `trace_id`.

Context before insight

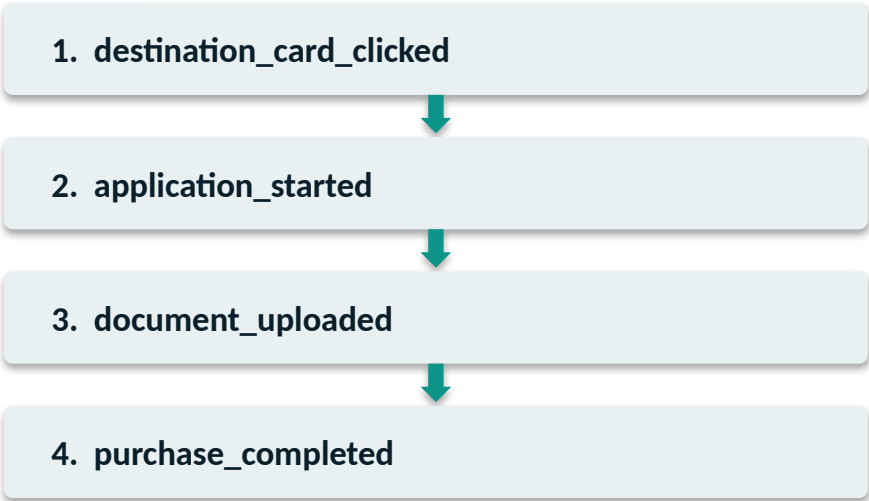
Analytics never runs on a stale snapshot. `context.updated` always precedes `insight.created`.

Chat is a thin client

The LLM brain picks tools; FastAPI owns state. Reloads don't lose a pending run.

ClickHouse data model

EXISTING FUNNEL (~2.5M rows)



+ supporting: search · scroll · auth · pay_now

Per feature: <feature>_events
Designed from the spec's NDJSON — types,
ORDER BY, partition, TTL — after approval.

PIPELINE META (auto-created)

schema_catalog / changelog	DDL, rationale, versions
context_snapshots	versioned business layer
context_changelog	diffs + contradictions
insights	cards + evidence + SQL
pending_runs	approval gate state
known_issues	linked from context
migration_journal	safe DDL apply
atlys.event_log	durable event backbone

Mapped to judging criteria

- | | | |
|----|--------------------------|---|
| 01 | Schema quality | Event-driven typing from sample NDJSON. Defensible ORDER BY / partition / TTL. MVs that earn their keep. Human approval before execute. |
| 02 | Insight quality | Playbook cuts across device, geo, funnel stage. Known-issue correlation. Charts & tables backed by stored SQL — PM prose, not DB dumps. |
| 03 | Context freshness | Context Agent versions snapshots + diffs on every schema change. Analytics always reads the latest snapshot. |
| 04 | Traceability | Langfuse dual-trace (chat + pipeline), cross-linked by session_id = run_id. meta.* and event_log carry trace_id. |
| 05 | Unseen spec | Zero per-spec hardcoding. Same interrogate → propose → approve → analyze path. Built for the sealed sixth spec. |

Feature spec in. Production schema + PM-ready insights out.

Agents instrument, analyze, and explain — fully traced.

- 1 Upload / pick a feature spec → "implement Express Checkout"
- 2 Agent interrogates gaps → proposes DDL + rationale
- 3 Approve in chat → CREATE TABLE + load + context reconcile
- 4 Insight card + charts & tables + evidence SQL + Langfuse link
- 5 Dashboard: changelog · context diff · event_log replay

Stack

ClickHouse Cloud • FastAPI / MCP • LibreChat • LLM (brain) • Langfuse • React